PLATFORM DOCUMENTATION · V1.0

Lead Intel

A lead-intelligence platform for local-business sales. It finds businesses on Google Maps, reads their websites, measures 64 named checks against them, scores what is wrong, decides what to sell them, and buys extra data only when an operator asks for it — one business at a time, with the price shown first.

BUILT BY	APPLICATION	STACK
AheadTech360	**Lead Intel (internal)**	**TypeScript · Next.js 16 · Supabase · Trigger.dev**

HOSTING	AUTOMATED TESTS
Vercel (bom1) · Supabase (ap-south-1)	**654 passing**

01 What the platform is

Cold outreach to local businesses fails for one reason above all others: the seller has nothing specific to say. Lead Intel exists to remove that problem. For every business in the list it answers three questions before anyone picks up the phone — **is this business worth calling**, **what exactly is wrong with their online presence**, and **what should we sell them** — and it answers them from measurements, each one traceable back to the page, listing or account it came from.

In non-technical terms

The operator types a trade and a place — “screen printing, Dallas”. The platform searches Google Maps across a grid of points, removes duplicates, then visits every website it found and reads it. It counts what is missing: no mobile layout, no phone number in the header, no reviews on the page, a contact form with fourteen fields, no tracking of any kind. It turns those counts into two scores, sorts the list so the best call is the first row, and writes the opening line of the call from one specific finding.

Everything above is free. Anything that costs money — a business's Google listing details, their Instagram following, whether they are running ads — is bought only for businesses the operator has ticked, on a screen that shows the price, the provider and the total first.

In technical terms

A TypeScript monorepo: a server-rendered Next.js 16 application for the operator, a Trigger.dev worker fleet for everything long-running, a Supabase Postgres database with row-level security, and a set of provider adapters (Google Places, Google PageSpeed, YouTube Data API, Apify, Google AI Studio) behind one interface each. Business logic — checks, scoring, ICP matching, deduplication, import parsing — lives in a pure package with no I/O, which is why it can carry 540 unit tests.

422
businesses in the system

17,943
findings recorded

347
website pages read in a browser

$5.39
total spent with all providers

> **The rule the whole product is built on: unknown is a third answer.** A business whose website could not be reached is not a business with a bad website. A business whose Instagram was never bought does not have zero followers. Every stored value can be *yes*, *no*, or *nobody has looked*, and the third never silently becomes one of the first two — not in a score, not in a filter, not in a chart.

02 The four stages

Every screen in the product belongs to exactly one of four stages. Two of them can cost money and two can never cost anything, and the interface says which is which at all times.

Stage 1
Sourcing
Getting businesses in: a Google Maps search for an ICP, a CSV import, or one typed by hand. CAN COST

Stage 2
Filtering
Narrowing to the customers you want: your ICPs, then the filters on the leads list. FREE

Stage 3
Analysis
Judging what is wrong: the crawler, 64 checks, your checklists, speed, reviews. FREE

Stage 4

Enrichment

Buying what is missing: Google listing, Instagram, ads — per business, after a confirm screen. PAID

The order matters operationally: filtering and analysis are free and repeatable, so the list is narrowed and judged *before* a single paid lookup is made. In the data so far, 422 businesses have produced 17,943 findings for a total provider spend of $5.39.

03 Feature inventory

Every screen currently in the product, and what it does.

Dashboard

- The four stages as columns, each holding its own figures: businesses in the list and where they came from; how many are worth calling and how many are in an ICP; how many have been analysed and how many are Priority 1; total spend and what has been bought.
- **What to do next** — only actions that actually need doing appear, each with a single button (analyse the unanalysed, call the Priority 1 leads, price the Instagram accounts nobody has looked up, log the first calls).
- Four breakdowns — priority, what to sell, website state, ICPs — where every row is a link that opens the leads list with that filter applied.
- Spend grouped by the company that actually invoices it, not by internal endpoint name.

Leads — the call list, in three steps

- **Step 1 · Choose:** search by name, town or ICP, plus one menu per filter — ICP, worth-calling view, claimed, priority, what to sell, website state, row colour, Google reviews, place, Instagram & ads, source, one imported file. Every choice becomes a removable chip with a live count (“176 of 422 businesses”), and the table never shifts when a menu opens.
- **Step 2 · Tick:** a fixed-column table, 50 rows a page, with row tints that encode presence: red = reviews but no website and no Instagram, blue = Instagram but no working website, peach = website but no Instagram, green = both.
- **Step 3 · Act:** one panel holding every action with its price and its billing provider — analyse (free), read pages in a browser (0.36¢ a site, Gemini), buy extra data (priced on the next screen), download CSV (free), plus two whole-list jobs. Buttons that need a selection are visibly switched off and say why on hover.

- Two filters — **Budget** and **Revenue** — are deliberately present and switched off, with the reason on hover: nobody publishes what a small private business earns or spends.

One business — four tabs

- **Overview:** why this lead, the opening line to use, an optional AI briefing, their Instagram and ads, and "everything we know" grouped by *who told us* — Google listing, their own website, away from their site — because those sources are not equally reliable.
- **Website pages:** one scorecard per page a real browser read, with screenshots of the home and product pages, the checklist verdicts for that page, and the model's read of it.
- **Checks & scores:** score by area, the biggest problems with their evidence, speed, every check with its verdict (wrong / fine / could not measure), and where every stored value came from.
- **Calls & history:** log a call in a panel (channel, result, objection, what you opened with, notes, meeting date), previous calls, and every job ever run on this business with its cost.

ICPs

- The list of ICPs in priority order, each showing its industry, the words it is searched by on Google Maps, the words and conditions that recognise one, and how many businesses are in it.
- Add or edit in a panel: name, industry, Google Maps search words, three separate word boxes (anywhere / their website / their Instagram), exclusions, follower range, review floor, required working website, places.
- A preview that counts what the ICP would take *before* saving, and a free re-match over the whole list.
- A standing count of businesses no ICP describes — kept, never deleted.

Checklists

- The operator's own definition of a good page: which page type, which industry, what to measure, the comparison and value, the weight, and the sentence to say on a call when it fails.
- Every rule is measured against the 347 pages already read, so the effect of a rule is visible before it is saved. 12 rules are live today.

Find businesses · Import · Imported files · Add by hand

- Search Google Maps for an ICP (all its words, one run each) or a single word; choose a place and radius; see the estimated number of requests and the cost range; set a spend cap and a business cap. Nothing is spent before the confirm screen.

- CSV import with loose column matching, a preview that separates new rows, rows that will merge into existing businesses, possible duplicates and unreadable rows, and a plan for what happens after import.
- Every uploaded file is kept with what it produced, and can be deleted.

Instagram · Data & spend · How it works

- Instagram: who has an account, who has been looked up, followers, Instagram's own category beside your ICP, and who is running ads.
- Data & spend: every source, what it gives, price per business, who bills it, and a ledger of what has actually been spent.
- How it works: every rule in the product in plain words, generated from the same constants the code uses.

Cross-cutting

- **Background jobs** appear at the top of every screen while they run, with progress, cost so far, elapsed time and a **Stop** button that cancels the platform run and marks the row stopped.
- **Hover help** on buttons and filters explains what pressing does and what it costs; a switched-off control explains why it is off.
- The whole application is server-rendered with **no client-side JavaScript**: filters, panels, tabs, ticks and pagination are all URLs and form submissions.

04 How a website is analysed

Website analysis runs in two layers. The first is free, fast and used on everything; the second is a real browser and is used where it earns its cost.

Layer 1 — our own crawler FREE

1. **Resolve and fetch.** Three DNS attempts, `robots.txt` respected via `robots-parser`, a pinned desktop Chrome user-agent and an `x-crawled-by` header. HTML is capped at 12 MB; a larger document is recorded as `too_big` rather than failing silently.
2. **Walk the site.** Home page first, then internal links classified into page types — home, services, products, pricing, about, contact — so a rule can address “the contact page” and mean it.
3. **Extract structure.** Parsed with `cheerio` into a JSON description of the page: headings, forms and their field counts, tap-to-call links, embedded reviews, images and alt text, schema markup, social links (scanned across the whole document, not only the footer), tracking scripts, booking and chat widgets, careers links.

4. **Store evidence.** Text digests, emails and phone numbers found on their own pages, the final URL, HTTP status, and the outcome of the crawl are written to `field_metadata` and `page_audits`, each with a source name and a timestamp.

Layer 2 — a real browser 0.36¢ A SITE

1. **Render.** Playwright 1.57 (Chromium) opens the page at phone width, auto-scrolls it so lazy content loads, and measures layout facts a parser cannot see: what is above the fold, whether the page scrolls sideways, tap-target sizes, how long until main content appears.
2. **Screenshot.** Home and product pages are captured, clipped to three viewports, and stored in a private Supabase Storage bucket; links to them are signed on demand and expire.
3. **Critique.** The measurements — never the raw page — are handed to Gemini, which writes what those measurements miss in a sentence an operator can say on a call. See §8.
4. **Re-score.** The business is scored again immediately, so a page read changes its priority in the same run.

Checklists — the operator's own definition of good

Both layers feed a checklist engine. A checklist rule names a page type, optionally an industry, a measurable field, a comparison, a value, a weight, and the sentence to say when it fails. Rules are evaluated against stored page structure, so editing a rule re-judges 347 already-read pages instantly and for free. A rule that cannot be evaluated — the page never rendered, the field was never captured — returns *not checkable* and is excluded from the page score rather than counted as a failure.

05 The check library

64 checks are defined, each with a stable identifier, a family, a weight and an explanation. About 51 produce verdicts on live data today; the remainder are recorded as *not measured* with a stated reason, because the data behind them is not published by anyone (see §9).

Check families, the area each feeds, and what they measure

FAMILY	IDS	AREA	WHAT IT MEASURES
W — Website	W1–W12	Website quality	Mobile layout, HTTPS, speed, modernity, broken images, sideways scroll, page weight, last-updated signals
K — Contact	K1–K8	Turning visitors into customers	Phone in the header, tap-to-call, address, opening hours, contact form length, call-to-action above the fold
S — Search	S1–S14	Being found on Google	Titles and meta, one main heading, schema markup, sitemap, indexability, Google listing completeness and category
R — Reputation	R1–R6	Reviews & reputation	Review count and rating, recency, whether reviews appear on their own site, review names
O — Off-site	O1–O8	Social media	Instagram and Facebook presence, follower scale, posting activity, YouTube channel and last upload
D — Demand capture	D1–D6	Ads & tracking	Analytics, pixels, tag managers, conversion tracking, live ads
N — Intent signals	N1–N10	(feeds the second score)	Evidence they already pay for things: booking tools, chat widgets, careers pages, paid themes, ad pixels

Every finding stores the evidence that produced it, so a verdict on a record can always be traced to the page and the measurement behind it. Current distribution across 422 businesses: **7,884 passed**, **3,909 fired**, **5,899 not measured**, 251 not applicable.

06 ICP rules and matching

An ICP — ideal customer profile — is the operator's own description of a customer they want. It does two jobs in one record: it is **searched for** on Google Maps, and it **recognises** businesses already in the list. Writing it once is what keeps those two halves from drifting apart.

What an ICP holds

Name and industry

What the operator calls this customer, and the trade it sits in. The industry is also what scopes checklist rules.

Search words

What a person would type into Google Maps. Several per ICP; Find businesses runs one search per word.

Word groups

Three separate lists: *anywhere* (business name, website text, domain, Instagram bio, Instagram category, handle, ad copy), *their website only*, *their Instagram only*.

Exclusions

Words that rule a business out, checked across every source the ICP can read — suppliers, equipment sellers and wholesalers use the same vocabulary as the customers wanted.

Conditions

Follower range, minimum Google reviews, a working website required, and places (towns or states).

Order and on/off

Lower order is checked first; the first ICP that matches wins; an ICP can be switched off without being deleted.

How matching works

1. ICPs are sorted by order, disabled ones removed.
2. **Conditions are asked first.** If an ICP requires a working website, a business whose site could not be reached is declined — and so is one nobody has ever looked at. Review floors and follower ranges behave the same way. *An unanswerable condition never passes.*
3. Exclusions are checked across every source that ICP can read, not only the source that matched.
4. Each filled word group is asked separately. `requireAll = false` means any group firing is enough; `true` means every filled group must fire — the only way to express “sells printing on their site *and* presents as a clothing brand on Instagram”.
5. Term matching requires a word boundary on at least one side, so `dtf` matches `bestpricedtf.com` and `dtfdallas.com` but not an accidental run of letters inside an unrelated word.
6. The first ICP to satisfy everything wins. The business stores the ICP's name, its id, and the time it was matched; the record shows which word matched and where it was found.
7. A business that no longer fits any ICP has it **cleared**, not left behind. A stale label looks like a decision somebody made.

> Matching reads only stored text and stored facts. It makes no network calls and costs nothing, which is what makes “edit an ICP, re-match everyone, look, edit again” a usable loop over hundreds of businesses.

Five ICPs are live today. 200 of 422 businesses sit in one; 222 match none and are kept in one place rather than deleted.

07 Scoring, priority and pitch

Two scores

Checks roll up into six areas — website quality, being found on Google, reviews and reputation, turning visitors into customers, social media, ads and tracking. Each area scores 0–100, where higher means more is wrong.

```
problem = 0.6 × (worst area) + 0.4 × (weighted average of all areas)
intent  = weighted roll-up of the N family (do they already pay for things?)
```

Weighting the worst area at 60% is deliberate: a business with one catastrophic problem is a better call than one that is mediocre everywhere, because the call has something specific to open with.

Presence override

If a business has no website, a dead one, or only a social page, the problem score is lifted to the presence floor and the recommendation is forced to **build them a website** — nothing else can be sold until there is one.

Priority

Tiers, from problem score ≥ 55 and intent score ≥ 35

TIER	PROBLEM	INTENT	MEANING
1	high	high	Lots wrong *and* they already spend money — call these first
2	low	high	Spends money, little wrong — a harder sell
3	high	low	Lots wrong, no sign they spend yet
4	low	low	Lowest priority

The spend gate

A business passes the gate at a problem score of **55** and no shared-domain warning. The gate decides where money *may* be spent; it never spends it. It is also the documented lever: if a run would cost too much, the cut-off is raised so fewer businesses qualify — the depth of analysis on those that do is never reduced.

What to sell them

The recommendation is the area that scored worst, mapped to one of six routes: build them a website, get them found on Google, fix their reviews, make their site convert, run their social media, set up ads and tracking. **A route with no finding to show is not offered at all** — the record says so, and the business is flagged for manual review instead of being given an unsupported pitch.

08 Where AI is used

> **Model:** Google Gemini `gemini-2.5-flash`, called through the Google AI Studio API from a single adapter in `packages/providers/src/gemini`. **Nothing about a business is decided by a model.** Scores, priorities, ICP membership and the recommended pitch are computed by code from measurements. AI is used in exactly three optional places, and each one is a suggestion or a rewording of numbers the system already holds.

1 · Writing an ICP from a description ≈0.1¢ PER USE

The operator describes the customer they want in plain English or Roman Urdu — “mujhe wo log chahiye jo apna clothing brand chalate hain aur khud print bhi karte hain”. The model is given that description, the ICPs that already exist so it does not duplicate them, and real sample text from the operator's own businesses so the vocabulary comes from how these businesses actually write. It returns a draft: name, industry, Google Maps search words, the three word groups, exclusions, and one line of reasoning per term.

Guardrails: the draft is never saved. It fills the form, a mechanical check flags terms that are too generic or too short to be safe, and the preview counts what the draft would actually catch before the operator presses Save. Conditions (reviews, followers, website, places) are never suggested — they are claims about facts, so the operator sets them deliberately.

2 · Critiquing a page that a browser read 0.36¢ A SITE

After Playwright renders a page, the model is handed the *measurements* — heading structure, form fields, what sits above the fold, tap-target sizes, load timings, the page's own text digest — and asked what those measurements miss, in a sentence that can be said on a call. It is explicitly not asked to browse, to fetch, or to state facts about the business. Its answer is stored beside the page it describes, never mixed into the findings, and the checklist verdicts remain the system's own.

3 · A plain-words briefing for one business ≈0.3¢ PER USE

On a lead record, the operator can ask for one paragraph summarising the findings already on screen, so they can read a briefing instead of scanning a list of 20 lines before dialling. The model receives the stored findings, the scores and the business's own website text. If the model's answer cannot be parsed, or it disagrees with the operator's ICP, the record says so and the underlying findings are shown unchanged.

The standing rules for AI in this system

- **It is shown measurements; it is never asked to find facts.** No model call decides whether a business has a website, how many reviews it has, or which ICP it belongs to.
- **Every call is priced and logged.** AI spend appears in the same ledger as every other provider, grouped under Google AI Studio (Gemini) on the dashboard.
- **Every call is optional.** The platform runs end to end with AI switched off; the three features above simply do not appear.
- **Failures are visible.** A refused, unparseable or quota-limited answer is reported to the operator with the reason — never replaced with a guess.

09 Data sources and cost

Free — runs on every business

SOURCE	WHAT IT GIVES	PROVIDER
Our own crawler	Pages, structure, emails, phone numbers, social links, text digests	Self-hosted (Trigger.dev machine time)
The 64 checks	Verdicts with evidence, in six areas plus intent	Own code
Scoring and ICP matching	Two scores, tier, recommended pitch, ICP membership	Own code
Google PageSpeed	Mobile performance, main-content timing	Google PageSpeed Insights API
YouTube channel	Channel presence and last upload (daily quota)	YouTube Data API v3

Paid — bought per business, after a confirm screen

SOURCE	WHAT IT GIVES	PER BUSINESS	BILLED BY	STATUS
Google listing details	Rating, review count, hours, category, trading status	3.5¢	Google Places API	Live
Instagram profile	Followers, posts, Instagram's category, bio, link	0.26¢	Apify	Live
Facebook / Instagram ads	Whether ads are running, how many, since when	11.6¢	Apify	Live
Page critique	The model's read of one rendered page	0.36¢	Google AI Studio (Gemini)	Live
Owner's mobile	A direct mobile number for the owner	73¢	Lusha	Needs a key
Website traffic estimate	Estimated monthly visits	0.12¢	DataForSEO	Needs a key

How spending is controlled

- **A ledger row is written before the call, not after.** `provider_calls` holds 326 rows totalling $5.39, and every figure the product shows about money is read from it.
- **Idempotency:** anything already bought for a business is free the second time — the confirm screen says how many of the selected businesses are already covered.
- **Spend caps:** a search carries a maximum spend and a maximum number of businesses; choosing an ICP with four search words splits the cap across the four runs rather than multiplying it.
- **Confirm screens:** no paid source is ever bought as a side effect of another action.

What cannot be bought at any price, and is documented as such:

- The list of who follows an Instagram account — Instagram publishes a follower count and never the list.
- What a business spends on advertising — Meta publishes spend only for political and issue ads.
- Whether a Google listing has been claimed — the field does not exist in the Places API.
- Budget and revenue for small private businesses — no provider publishes it, which is why both filters ship switched off with that reason on hover.

Compliance note carried in the product: in the US, TCPA treats mobile numbers as residential even when they belong to a business owner, and the B2B exemption covers business landlines. That is why the owner's-mobile source is built, priced and left switched off pending a compliance review rather than quietly enabled.

10 Technical architecture

Shape

```
lead-intel/                    npm workspaces monorepo, TypeScript 5.6 (strict)
├─ apps/
│  ├─ web/      Next.js 16.3 · React 19.2 · App Router · server components
│  └─ jobs/     Trigger.dev 4.5 worker: 13 tasks · Drizzle ORM · Playwright
└─ packages/
   ├─ core/     pure business logic — checks, scoring, ICP matching,
   │            deduplication, import parsing, outreach lines   (no I/O)
   ├─ providers/ one adapter per external service, each with its own errors
   ├─ db/       Drizzle schema + client (worker side)
   └─ shared/   enums, constants, prices, thresholds — one source of truth
```

Languages and runtimes

TypeScript 5.6

Every line of application code, strict mode, no `any` in the public surface of a package.

SQL (PostgreSQL 15)

Schema, row-level security policies, indexes and hand-written migrations.

HTML + CSS

Hand-written, no UI framework and no CSS framework: one token file, semantic colour, and components built from it.

Node.js 20+

Runtime for both the web app's server side and the worker (currently Node 22 locally).

Python 3

Only for the design-wireframe generator that produces the product's low-fidelity screens.

Database — Supabase PostgreSQL (ap-south-1)

17 tables, all under row-level security. The core of the model:

TABLE	COLS	HOLDS
`businesses`	36	One row per business: identity, presence, scores, tier, route, ICP, claim state, origin
`identity_keys`	7	place_id, domain, phone, name+location — the keys deduplication matches on, ranked
`field_metadata`	14	Every stored fact with its source and when it was verified — the audit trail
`findings`	16	17,943 check verdicts with evidence, outcome and weight
`page_audits`	22	347 rendered pages: type, structure JSON, layout JSON, screenshot path, critique
`page_checklists`	13	The operator's own page rules, scoped by page type and industry
`icps`	20	ICP definitions: words, groups, exclusions, conditions, order
`runs` + `run_grid_cells`	15 / 11	Discovery searches and the geographic grid each one covered, including cells that saturated and split
`provider_calls`	10	The money ledger: endpoint, business, cost, status — written before each paid call
`job_runs`	14	Progress, cost and outcome of every background job, linked to its Trigger.dev run
`outcomes`	11	Logged calls: channel, result, objection, what was opened with, notes
`import_batches`, `merge_snapshots`, `exclusions`, `suppression_hashes`, `user_roles`	9 / 9 / 7 / 5 / 3	Uploaded files, reversible merges, do-not-contact lists, roles

Connections and external services

CONNECTION	PROTOCOL / SDK	USED FOR
Web → Supabase	`@supabase/ssr` + PostgREST over HTTPS, user's JWT	All reads and writes the operator performs, under RLS as that user
Worker → Supabase	Drizzle ORM over the Postgres pooler, service role	Bulk writes from jobs — crawling, scoring, matching, enrichment
Web → Trigger.dev	`@trigger.dev/sdk` (`tasks.trigger` , `runs.cancel`)	Enqueueing and stopping jobs; the web app never does long work itself
Worker → Google Places	REST (Text Search, Place Details)	Discovery and listing details
Worker → PageSpeed / YouTube	REST	Mobile performance; channel presence
Worker → Apify	REST actor runs	Instagram profiles; Facebook/Instagram ad presence
Worker → Google AI Studio	REST (Gemini 2.5 Flash, JSON mode)	The three AI features in §8
Worker → the open web	fetch + Playwright Chromium	Crawling and rendering business websites
Storage	Supabase Storage, private bucket, signed URLs	Page screenshots

Background jobs

13 Trigger.dev tasks, each idempotent and each reporting progress into `job_runs` : `discovery` , `analyse-run` , `audit-website` , `audit-site` , `audit-site-run` , `audit-run` , `pagespeed` , `pagespeed-run` , `youtube` , `youtube-run` , `match-icps-run` , `score-run` , `enrich-run` . Long passes run in batches, check a stop flag between batches, and write partial results — a stopped job keeps everything it had already done.

Application design choices

- **Zero client-side JavaScript.** Every interaction is a link, a GET form or a server action: filter menus are `<details>` , panels and tabs are URLs, ticking rows is a form. The result is a product that works on a slow connection and has no hydration cost.
- **One predicate, two screens.** The leads list and the buy screen share `matchesFilters` , so “buy for all 176 shown” can never price a different set than the one on screen.

- **One source of truth for constants.** Prices, thresholds, area weights and enums live in `packages/shared` and are read by the worker, the web app and the “How it works” page alike.

11 Engineering practice

654
automated tests passing

540
in the pure logic package

21
applied SQL migrations

0
client-side JS bundles for UI logic

- **Testing:** Vitest. The `core` package is pure, so every check, score, tier boundary, ICP condition, dedup rule and CSV edge case is tested without a database. Provider adapters are tested against recorded responses, including failure shapes — quota exhausted, rate limited, empty body with HTTP 200.
- **Type safety:** strict TypeScript across the monorepo; database types flow from the Drizzle schema; enums and prices are shared constants rather than string literals.
- **Migrations:** hand-written SQL, applied through Supabase, kept in `packages/db/drizzle` with the reasoning in comments — including why a column was renamed rather than dropped.
- **Error philosophy:** a step that could not run never looks like a step that ran fine. Failures are written to the record with their reason and surfaced on the screen the operator pressed the button on.
- **Accessibility and contrast:** body text at 12.9:1 and small print at 9:1 against the page — measured, not eyeballed, after the operator reported grey text twice.
- **Documentation in place:** non-obvious decisions are commented where they live; the “How it works” screen renders the product's own rules for the operator.

12 DevOps and operations

Environments

	WEB	WORKER	DATABASE
Local	`next dev` on :3000	`trigger dev`, development key	Shared Supabase project
Production	Vercel, region `bom1`	Trigger.dev cloud, production key	Supabase `ap-south-1`

Trigger.dev environments are sealed from each other — a development key can only reach the local worker — but both share one database, so work started locally is visible in the deployed app. That is documented for the operator, because it also means local testing changes production data.

Deployment

- **Web:** push to `main` on GitHub → Vercel builds and promotes. The build runs the workspace package build first (`prebuild`), so a type error in shared code fails the deploy rather than reaching production.
- **Worker:** `npm run deploy --workspace=@lead-intel/jobs` builds a container image (Playwright pinned to 1.57.0 for the image's browser check) and rolls out a new version on Trigger.dev.
- **Database:** migrations applied deliberately and separately, before the code that needs them.
- **Rollback:** Vercel keeps previous deployments; Trigger.dev keeps previous worker versions; migrations are written to be additive where possible.

Performance engineering

The most recent operational fix is a good example of the class of problem this stack has: pages took 2–3 seconds because the Vercel functions ran in `iad1` (Washington) while the database sits in `ap-south-1` (Mumbai) — every query crossed the world. Pinning the functions to `bom1` through `vercel.json` put server work at roughly 100–160 ms. The leads list was separately paginated to 50 rows, cutting a 21,000-pixel page to a sixth of its height.

Secrets and security

- Row-level security on every table; the web app acts as the signed-in user, never as an admin.
- The Supabase service-role key exists only in the worker's environment and in Trigger.dev's production environment — never in the web app, never in the browser, never in the

repository.

- `.env` files are git-ignored by name as well as by pattern; the example file carries names only.
- Screenshots live in a private bucket and are served through short-lived signed URLs.
- Recordings and other large media are ignored by the repository.

Observability and control

- Every job writes progress, a running cost and a human sentence into `job_runs`; the operator sees it on any screen and can stop it.
- Trigger.dev's own dashboard carries logs and run history per task.
- The money ledger is the single source for every cost figure in the product — nothing is estimated in the interface.

13 Roles and skills involved

What building and running this platform actually requires, in the terms a company hiring for it would use.

Full-stack engineering

TypeScript across the whole stack; Next.js App Router with server components and server actions; hand-written CSS with a token system; PostgreSQL schema design; ORM and query work; REST integration with six external providers.

Backend / distributed jobs

Idempotent background tasks, batching, cancellation, progress reporting, retry and quota handling, provider failure taxonomy, ledger-backed spend control.

Data engineering

Entity resolution and merge rules across four identity keys, evidence-preserving schema design, CSV ingestion with duplicate detection, a scoring model with documented weights and thresholds.

DevOps

Vercel and Trigger.dev deployments, region placement against database latency, environment and secret separation, container image builds for

the worker, migration sequencing, rollback paths.

Web scraping and rendering

Politeness and robots handling, resilient fetching, HTML parsing, headless Chromium rendering, layout measurement, screenshot capture and private storage.

Applied AI

Prompt design with strict JSON contracts, defensive parsing, cost accounting per call, and — the part that matters most here — drawing the line between what a model may suggest and what only measurement may decide.

Product and UX

Wireframes for 27 screens, a three-step worklist, a no-JavaScript interaction model, hover help on every control, and an explicit stance that unknown data must never be shown as a negative answer.

Compliance awareness

Outreach rules per channel, TCPA exposure on mobile numbers, and a documented list of data that cannot be obtained legally or at all.

14 Current state and roadmap

Live today

AREA	STATE
Sourcing — Google Maps, CSV, manual	Live; 12 searches run so far
Crawling, 64 checks, scoring, routes	Live; 17,943 findings across 422 businesses
Browser page reads and critiques	Live; 347 pages read
ICPs with conditions and search words	Live; 5 ICPs, 200 businesses matched
Checklists	Live; 12 rules measured against every read page
Enrichment — Places, Instagram, Meta ads	Live; 326 paid calls, $5.39 total
Call logging and outcome capture	Live; the feedback loop that will tune the weights

Built, priced, and deliberately switched off

- **Owner's mobile (Lusha, 73¢):** integrated in the pricing model and the confirm screen, disabled pending a compliance review.
- **Website traffic estimates (DataForSEO, 0.12¢):** awaiting a key.
- **Budget and revenue filters:** visible, switched off, with the reason on hover — no provider publishes the data.

Next

- Enough logged call outcomes to tune the check weights and the gate against real replies rather than estimates.
- Finding a business's Google listing from a name alone, for imported rows that arrive without one — currently 123 businesses cannot be scored on the reputation and local-search areas because a spreadsheet carries no `place_id`.
- Rolling the remaining screens onto the new three-step interaction model.

> **The measure of this platform is not how much data it holds.** It is that every number on a record can be traced to the page, listing or account it came from, that every pound spent was authorised on a screen that showed the price first, and that nothing the system does not know is ever presented as something it does.

Lead Intel · platform documentation v1.0 · AheadTech360 · figures taken from the production database on 21 September 2026.